

Exercício: API da Lions Bet (Casa de Apostas com Admin)

Turma: LionsDev

Tópicos: Boilerplate, autorização por papel (`role` / `tipo`), `tipo` dentro do JWT, rotas públicas × rotas de usuário × rotas de admin, middleware `apenasAdmin`, carteira/saldo, dois recursos com `ObjectId` / `ref`, regra que cruza coleções (liquidação de apostas) e status codes (`401` × `403`).

Nível: o mais difícil do módulo. Faça os outros antes (Tarefas, Petshop, Academia, Finanças, Biblioteca). Aqui, pela primeira vez, você vai mexer na autenticação do boilerplate para criar o admin.

1. Contexto

Você vai construir a Lions Bet, uma casa de apostas esportivas. A API tem três tipos de visitante:

Quem	O que pode fazer
Visitante anônimo (sem login)	Ver a vitrine de eventos abertos e suas odds. Cadastrar-se e logar.
Usuário logado (<code>tipo: "usuario"</code>)	Depositar saldo, apostar em eventos abertos e ver as próprias apostas.
Administrador (<code>tipo: "admin"</code>)	Criar eventos, ajustar odds, encerrar um evento definindo o resultado (isso paga os ganhadores automaticamente) e ver todas as apostas e todos os usuários.

Até agora, os exercícios usavam só o padrão "dono do recurso via token", sem papéis. Aqui entra a **autorização**: além de saber quem está logado, a API precisa decidir o que essa pessoa pode fazer. É a diferença entre `401 Unauthorized` (não sei quem é você) e `403 Forbidden` (sei quem é você, mas você não pode fazer isso).

2. Ponto de Partida: o Boilerplate

Partimos do boilerplate LionsDev: <https://github.com/nicolassmotta/boilerplate-lions-dev.git>

1. Clone, `npm install`, crie o `.env` (`MONGO_URI` , `JWT_SECRET` , `JWT_EXPIRES_IN` , `BCRYPT_SALT_ROUNDS`) e suba o servidor.
2. Já estão prontos: camada de `Usuario` , cadastro/login com `bcrypt/JWT`, middleware `autenticar` (preenche `req.usuario`), `criarErro` e o middleware de erro.

Diferente dos outros exercícios, aqui você vai editar arquivos do boilerplate (model do usuário, geração do token, middleware de autenticação). Em projeto real, é assim que uma autenticação passa a suportar papéis.

3. Autenticação × Autorização

Autenticação -> "Quem é você?" -> middleware `autenticar` (já existe)
 Autorização -> "O que você pode fazer?" -> middleware `apenasAdmin` (você vai criar)

O boilerplate já autentica: valida o token e preenche `req.usuario` . Faltava autorizar. Para isso, o token precisa levar o papel do usuário (`tipo`), e um middleware novo (`apenasAdmin`) barra quem não é admin nas rotas restritas.

Os três níveis de proteção que você vai montar:

Nível	Middlewares na rota	Exemplo
Pública	(nenhum)	<code>GET /api/eventos</code> (vitrine)
Usuário	<code>autenticar</code>	<code>POST /api/apostas</code> (apostar)
Admin	<code>autenticar</code> , depois <code>apenasAdmin</code>	<code>POST /api/eventos</code> (criar evento)

4. Regras do Sistema

1. `tipo` nunca vem do body. No cadastro, todo mundo começa com `tipo: "usuario"` . Se a API aceitar `tipo` do body, qualquer pessoa se cadastra como admin.
2. `apenasAdmin` roda depois de `autenticar` . Ele depende de `req.usuario.tipo` , que só existe depois que o token foi validado.
3. `401` × `403` : sem token / token inválido → `401` . Token válido mas sem permissão → `403` .

4. **Aposta tem dono via token** (`req.usuario.id`): o usuário só vê e cria as próprias apostas. O admin vê todas.
5. **Saldo**: não deixe apostar sem saldo nem com valor `<= 0` . Debite ao apostar; credite só ao liquidar.
6. **Listagem nunca dá 404**: sem registros, devolva array vazio.

5. Etapa 0 — Autenticação com Papéis

Esta é a etapa nova do exercício: quatro edições no boilerplate e um middleware novo.

5.1 Adicionar `tipo` e `saldo` ao `Usuario` (`src/models/usuario.model.js`)

Acrescente dois campos ao Schema existente:

```
tipo: {
  type: String,
  enum: ["usuario", "admin"],
  default: "usuario",
},
saldo: {
  type: Number,
  default: 0,
  min: [0, "0 saldo não pode ser negativo."],
},
```

5.2 Colocar `tipo` dentro do token (`src/services/auth.service.js`)

Na função `gerarToken` , inclua o `tipo` nos dados do token:

```
const dadosDoToken = {
  id: usuario._id.toString(),
  email: usuario.email,
  tipo: usuario.tipo, // <- novo
};
```

No `cadastrar` , crie o usuário com `tipo: "usuario"` ou não passe `tipo` e deixe o `default` agir. Só não repasse `req.body.tipo` .

5.3 Ler `tipo` ao validar o token (`src/middlewares/autenticacao.middleware.js`)

Onde o middleware monta `req.usuario`, adicione o `tipo`:

```
req.usuario = {  
  id: dadosDoToken.id,  
  email: dadosDoToken.email,  
  tipo: dadosDoToken.tipo, // <- novo  
};
```

5.4 Criar o middleware `apenasAdmin` (`src/middlewares/apenasAdmin.middleware.js`)

```
import criarErro from "../utils/criarErro.js";  
  
// Roda SEMPRE depois de autenticar, então req.usuario já existe.  
function apenasAdmin(req, res, next) {  
  if (!req.usuario || req.usuario.tipo !== "admin") {  
    return next(criarErro("Acesso restrito a administradores.", 403));  
  }  
  return next();  
}  
  
export default apenasAdmin;
```

Como criar o primeiro admin? Ninguém vira admin pela API (regra 1). Cadastre um usuário normal. Depois, no MongoDB Atlas, edite o documento e troque `"tipo": "usuario"` por `"tipo": "admin"`. Faça login de novo para receber um token com `tipo: "admin"`.

Crerios de aceite da Etapa 0: depois de logar, o token decodificado contém `tipo`; `req.usuario.tipo` chega nos controllers; chamar uma rota de admin com token de usuário comum → `403`.

6. Etapa 1 — Models do Domínio (`Evento` e `Aposta`)

6.1 `Evento` (`src/models/evento.model.js`) — criado pelo admin

- **mandante** : **String** , obrigatório, **trim** (ex.: **"Brasil"**).
- **visitante** : **String** , obrigatório, **trim** (ex.: **"Argentina"**).
- **oddMandante** : **Number** , obrigatório, **min**: [**1.01**, **"Odd inválida."**] .
- **oddEmpate** : **Number** , obrigatório, **min**: [**1.01**, **"Odd inválida."**] .
- **oddVisitante** : **Number** , obrigatório, **min**: [**1.01**, **"Odd inválida."**] .
- **status** : **String** , **enum**: [**"aberto"**, **"encerrado"**] , **default**: **"aberto"** .
- **resultado** : **String** , **enum**: [**"mandante"**, **"empate"**, **"visitante"**] , opcional (só é definido ao encerrar).
- **criadoPor** : **ObjectId** , **ref**: **"Usuario"** (o admin que criou).
- **timestamps**: **true** .

6.2 Aposta (`src/models/aposta.model.js`) — criada pelo usuário

- **usuario** : **ObjectId** , **ref**: **"Usuario"** , obrigatório (o dono, via token).
- **evento** : **ObjectId** , **ref**: **"Evento"** , obrigatório.
- **palpite** : **String** , obrigatório, **enum**: [**"mandante"**, **"empate"**, **"visitante"**] .
- **valor** : **Number** , obrigatório, **min**: [**0.01**, **"0 valor deve ser maior que zero."**] (quanto foi apostado).
- **oddNaAposta** : **Number** , obrigatório (a API copia a odd do palpite no momento da aposta).
- **retornoPotencial** : **Number** (a API calcula: **valor * oddNaAposta**).
- **status** : **String** , **enum**: [**"pendente"**, **"ganha"**, **"perdida"**] , **default**: **"pendente"** .
- **timestamps**: **true** .

Por que `oddNaAposta` ? O admin pode mudar as odds enquanto o evento está aberto. A aposta guarda a odd do momento em que foi feita; sem isso, o pagamento mudaria depois. Esse é o conceito de *snapshot* do preço.

7. Etapa 2 — Repositories

7.1 Estender `UsuarioRepository` (`src/repositories/usuario.repository.js`)

Adicione uma função para mexer no saldo de forma incremental:

```
async function ajustarSaldo(idUsuario, valor) {
  // $inc soma (valor positivo credita, valor negativo debita) de forma
  atômica.
```

```
return Usuario.findByIdAndUpdate(
  idUsuario,
  { $inc: { saldo: valor } },
  { new: true, runValidators: true }
);
}
```

Lembre de incluir `ajustarSaldo` no objeto exportado `UsuarioRepository`.

Conceito novo: `$inc` (somar/subtrair direto no banco)

Para mexer em dinheiro, não leia o saldo, some em JavaScript e salve de volta. São duas idas ao banco e, se duas requisições rodarem juntas, uma sobrescreve a outra. O operador `$inc` manda o próprio MongoDB fazer a conta:

```
// credita 50 no saldo
Usuario.findByIdAndUpdate(id, { $inc: { saldo: 50 } }, { new: true });
// debita 50 do saldo (valor negativo)
Usuario.findByIdAndUpdate(id, { $inc: { saldo: -50 } }, { new: true });
```

Valor positivo credita, negativo debita. Por isso o `ajustarSaldo(id, valor)` serve para os dois casos: depósito (`+valor`), aposta (`-valor`) e prêmio (`+retornoPotencial`).

Conceito novo: `.save()` (ler, mudar, gravar)

Nos repositories de Evento e Aposta você vai usar `documento.save()` em vez de `findByIdAndUpdate`. Use `.save()` quando precisar buscar o documento, alterar um ou mais campos e gravar. É o caso de encerrar um evento (mudar `status` e `resultado`) e de liquidar uma aposta (mudar `status`):

```
const evento = await EventoRepository.buscarPorId(idEvento);
evento.status = "encerrado";
evento.resultado = "mandante";
await evento.save(); // grava as duas mudanças de uma vez
```

7.2 EventoRepository (`src/repositories/evento.repository.js`)

- `criar(dados) → Evento.create(dados)`.

- `listarAbertos()` → `Evento.find({ status: "aberto" }).sort({ createdAt: -1 })` .
- `listarTodos()` → `Evento.find().sort({ createdAt: -1 })` .
- `buscarPorId(idEvento)` → `Evento.findById(idEvento)` .
- `salvar(evento)` → `evento.save()` (usado para ajustar odds e para encerrar).

7.3 ApostaRepository (`src/repositories/aposta.repository.js`)

- `criar(dados)` → `Aposta.create(dados)` .
- `listarPorUsuario(idUsuario)` → `Aposta.find({ usuario: idUsuario }).sort({ createdAt: -1 })` .
- `listarPorEvento(idEvento)` → `Aposta.find({ evento: idEvento })` .
- `listarTodas()` → `Aposta.find().sort({ createdAt: -1 })` .
- `buscarPorIdDoDono(idAposta, idUsuario)` → `Aposta.findOne({ _id: idAposta, usuario: idUsuario })` .
- `salvar(aposta)` → `aposta.save()` (usado na liquidação).

8. Etapa 3 — Services (onde mora a regra de negócio)

8.1 Carteira: estender `UsuarioService` (`src/services/usuario.service.js`)

- `verCarteira(idDoUsuario)` : busca o usuário e retorna `{ saldo: usuario.saldo }` . Se não existir → `404` .
- `depositar(idDoUsuario, valor)` :
 1. Se `!valor || valor <= 0` → `criarErro("O valor do depósito deve ser maior que zero.", 400)` .
 2. `UsuarioRepository.ajustarSaldo(idDoUsuario, valor)` (positivo → credita).
 3. Retorne `{ saldo }` atualizado.
- `listarTodos()` : o boilerplate já tem `listarUsuarios()` ; reaproveite na rota de admin.

8.2 `EventoService` (`src/services/evento.service.js`)

- `criar(idAdmin, dados)` : monta o objeto com `criadoPor: idAdmin` e cria o evento (já nasce `aberto`).
- `listarAbertos()` : vitrine pública.
- `listarTodos()` : visão do admin (abertos + encerrados).
- `buscarPorId(idEvento)` : se `null` → `criarErro("Evento não encontrado.", 404)` .

- **atualizarOdds(idEvento, dados)** : busque; se **null** → **404** ; só permita editar se **status === "aberto"** (senão **criarErro("Evento já encerrado.", 400)**); aplique as odds e **salvar** .
- **encerrar(idEvento, resultado)** : a parte principal do exercício, porque cruza 3 coleções:
 1. Busque o evento. Se **null** → **404** . Se já **"encerrado"** → **criarErro("Evento já encerrado.", 400)** .
 2. Valide **resultado** no **enum** (**mandante** / **empate** / **visitante**); fora disso → **400** .
 3. Pegue todas as apostas do evento (**ApostaRepository.listarPorEvento**).
 4. Para cada aposta **pendente** :
 - Se **aposta.palpite === resultado** : **aposta.status = "ganha"** e credite o ganho com **UsuarioRepository.ajustarSaldo(aposta.usuario, aposta.retornoPotencial)** .
 - Senão: **aposta.status = "perdida"** .
 - **ApostaRepository.salvar(aposta)** .
 5. Atualize o evento: **status = "encerrado"** , **resultado = resultado** , e **salvar** .
 6. Retorne um resumo: **{ totalApostas, ganhadoras, perdedoras, totalPago }** .

8.3 ApostaService (src/services/aposta.service.js)

apostar(idDoUsuario, dados) , com **dados = { evento, palpite, valor }** :

1. Busque o evento por id. Se **null** → **404** .
2. Se **evento.status !== "aberto"** → **criarErro("As apostas para este evento estão encerradas.", 400)** .
3. Valide **palpite** (precisa ser **mandante** / **empate** / **visitante**); senão → **400** .
4. Se **!valor || valor <= 0** → **criarErro("O valor da aposta deve ser maior que zero.", 400)** .
5. Busque o usuário (**UsuarioRepository.buscarPorId**). Se **usuario.saldo < valor** → **criarErro("Saldo insuficiente.", 400)** .
6. Guarde a odd do palpite escolhido: mapeie **mandante** → **oddMandante** , **empate** → **oddEmpate** , **visitante** → **oddVisitante** .
7. **retornoPotencial = valor * oddNaAposta** (arredonde para 2 casas).
8. Debite: **UsuarioRepository.ajustarSaldo(idDoUsuario, -valor)** .
9. Crie a aposta com **usuario: idDoUsuario** , **evento** , **palpite** , **valor** , **oddNaAposta** e **retornoPotencial** (status nasce **pendente**).

Outras funções:

- `listarMinhas(idDoUsuario) : ApostaRepository.listarPorUsuario(...)` .
- `buscarMinha(idDoUsuario, idAposta) : buscarPorIdDoDono(...)` ; se `null` → `404` .
- `listarTodas()` : para o admin, `ApostaRepository.listarTodas()` .

Critérios de aceite: apostar debita o saldo na hora; apostar sem saldo → `400` ; apostar em evento encerrado → `400` ; ao encerrar com `resultado` igual ao `palpite` , o saldo do ganhador sobe exatamente em `retornoPotencial` ; perdedores não recebem nada; encerrar duas vezes → `400` (não paga em dobro).

9. Etapa 4 — Controllers e Etapa 5 — Routes (os três níveis)

Crie os controllers no padrão de sempre (`try/catch` + `next(error)`), lendo `req.usuario.id` , `req.params.id` e `req.body` .

9.1 `evento.routes.js` : rotas públicas e de admin (middleware por rota)

Aqui não use `router.use(authenticar)` global, porque parte das rotas é pública. Aplique os middlewares rota a rota.

- `GET /` → `EventoController.listarAbertos` (pública).
- `GET /:id` → `EventoController.buscarPorId` (pública).
- `POST /` → `autenticar, apenasAdmin, EventoController.criar` (admin).
- `PATCH /:id` → `autenticar, apenasAdmin, EventoController.atualizarOdds` (admin).
- `PATCH /:id/encerrar` → `autenticar, apenasAdmin, EventoController.encerrar` (lê `req.body.resultado` , admin).

9.2 `aposta.routes.js` : rotas de usuário e uma de admin

- `router.use(authenticar)` no topo (tudo exige login).
- `POST /` → `ApostaController.apostar` (`201` , usuário).
- `GET /` → `ApostaController.listarMinhas` (`200` com `{ apostas }` , usuário).
- `GET /:id` → `ApostaController.buscarMinha` (`200` , usuário).
- `GET /admin/todas` → `apenasAdmin, ApostaController.listarTodas` (admin).

9.3 Carteira + admin no `usuario.routes.js` (boilerplate)

Adicione, abaixo das rotas de perfil já existentes:

- `GET /carteira` → `autenticar, UsuarioController.verCarteira` (usuário).
- `POST /carteira/deposito` → `autenticar, UsuarioController.depositar` (lê `req.body.valor`, usuário).
- `GET /` → `autenticar, apenasAdmin, UsuarioController.listarTodos` (admin, lista todos os usuários).

Cuidado com a ordem: declare `/carteira` e `/carteira/deposito` antes de qualquer `/:id`, se você tiver alguma.

9.4 Registrar no `src/app.js` (antes do middleware 404)

```
import eventoRoutes from "../routes/evento.routes.js";
import apostaRoutes from "../routes/aposta.routes.js";

app.use("/api/eventos", eventoRoutes);
app.use("/api/apostas", apostaRoutes);
```

10. Rotas Finais

Método	Rota	Proteção	Descrição
POST	<code>/api/auth/cadastro</code>	Pública	Cadastro (nasce <code>tipo: "usuario"</code>)
POST	<code>/api/auth/login</code>	Pública	Login (token traz <code>tipo</code>)
GET	<code>/api/eventos</code>	Pública	Vitrine: eventos abertos e odds
GET	<code>/api/eventos/:id</code>	Pública	Ver um evento
POST	<code>/api/eventos</code>	Admin	Criar evento
PATCH	<code>/api/eventos/:id</code>	Admin	Ajustar odds (só se aberto)
PATCH	<code>/api/eventos/:id/encerrar</code>	Admin	Encerrar + resultado → paga ganhadores
GET	<code>/api/usuarios/carteira</code>	Usuário	Ver meu saldo
POST	<code>/api/usuarios/carteira/deposito</code>	Usuário	Depositar saldo
POST	<code>/api/apostas</code>	Usuário	Apostar (debita saldo)
GET	<code>/api/apostas</code>	Usuário	Minhas apostas

Método	Rota	Proteção	Descrição
GET	<code>/api/apostas/:id</code>	Usuário	Ver uma aposta minha
GET	<code>/api/apostas/admin/todas</code>	Admin	Todas as apostas
GET	<code>/api/usuarios</code>	Admin	Todos os usuários

11. Fluxo de Testes (use Postman)

Preparação dos papéis

1. Cadastre a Ana (`POST /api/auth/cadastro`). Ela será usuária.
2. Cadastre o Léo, vá ao MongoDB Atlas e troque o `tipo` de Léo para `"admin"` . Faça `POST /api/auth/login` com o Léo para pegar um token de admin.

Como admin (Léo)

3. `POST /api/eventos` com `Brasil x Argentina` e odds `{ oddMandante: 2.0, oddEmpate: 3.0, oddVisitante: 3.5 }` → `201` . Guarde o `_id` do evento.
4. `GET /api/eventos` sem token → `200` (rota pública mostra o evento).

Como usuária (Ana)

5. `POST /api/eventos` com o token da Ana → `403` (não é admin).
6. `GET /api/usuarios/carteira` → `200` , `saldo: 0` .
7. `POST /api/usuarios/carteira/deposito` com `{ "valor": 100 }` → `200` , `saldo: 100` .
8. `POST /api/apostas` com `{ "evento": "<id>", "palpite": "mandante", "valor": 50 }` → `201` , `oddNaAposta: 2.0` , `retornoPotencial: 100` . Confira a carteira: `saldo: 50` .
9. `POST /api/apostas` com `{ "valor": 1000 }` → `400` (saldo insuficiente).
10. `GET /api/apostas` → só as apostas da Ana.

Liquidação (admin Léo)

11. `PATCH /api/eventos/<id>/encerrar` com `{ "resultado": "mandante" }` → `200` , resumo com `ganhadoras: 1` , `totalPago: 100` .
12. Ana consulta `GET /api/usuarios/carteira` → `saldo: 150` (50 que sobrou + 100 do prêmio). A aposta dela agora está `ganha` .
13. `PATCH /api/eventos/<id>/encerrar` de novo → `400` (já encerrado, não paga em dobro).
14. `POST /api/apostas` no evento já encerrado → `400` .

Permissões

15. `GET /api/usuarios` (todos) com token da Ana → `403` ; com token do Léo → `200` .
 16. Qualquer rota de usuário/admin sem token → `401` .
-

12. Desafios Bônus

1. **Empate:** crie um segundo usuário que aposta em `"visitante"` no mesmo evento. Encerre com `"empate"` e confirme que ninguém recebeu (as duas apostas ficam `perdida`).
 2. `.populate()` em `GET /api/apostas` para trazer `mandante` / `visitante` / `status` do evento junto de cada aposta.
 3. **Relatório do admin:** em `GET /api/eventos/admin/relatorio` , calcule em JavaScript e retorne total apostado na casa, total já pago em prêmios e o "lucro da casa" (apostado – pago) considerando só eventos encerrados.
 4. **Limite de aposta:** bloqueie apostas acima de `R$ 1.000,00` por aposta com `400` .
 5. **Atomicidade:** discuta e tente resolver o problema de duas apostas simultâneas debitando o mesmo saldo. Por que o `$inc` ajuda e onde ainda há risco?
-

LionsDev • Professor Nicolas Cardoso Motta

Exercício: Lions Bet, autorização por papel com o Boilerplate - Módulo 09